



COMPSCI683: Artificial Intelligence

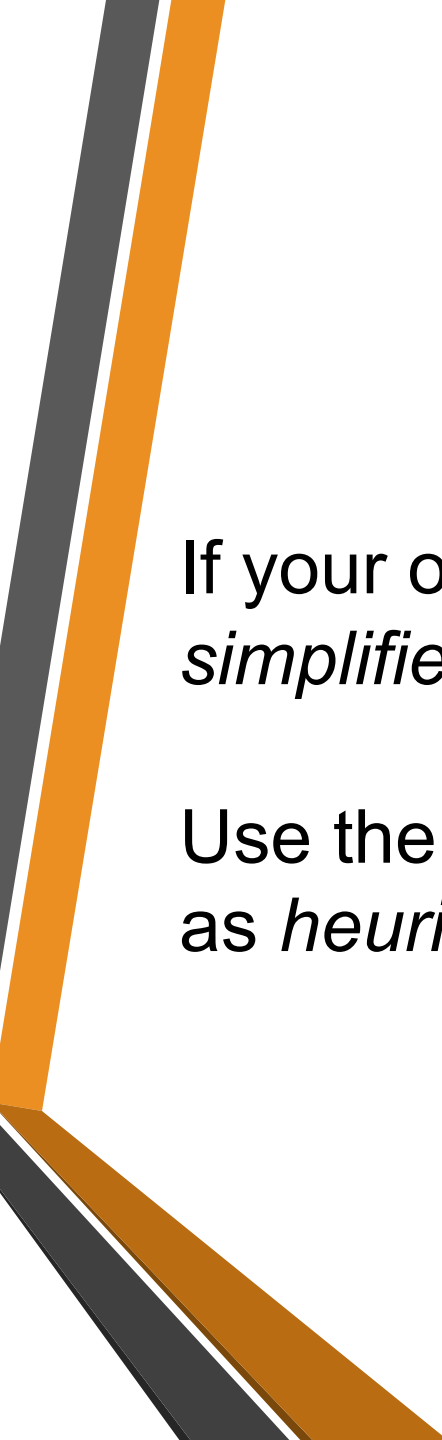
Instructor: Yair Zick

Today's slides are largely based on a tutorial by Robert Holte

I took them from Shlomo 😊

Today's lecture

- Where do heuristics come from?
- Generating heuristics using abstraction
- Using abstraction to speedup search
- Hierarchical search techniques
- Abstractions based on pattern databases



The Big Idea

If your original problem is hard, create a *simplified* version of your problem.

Use the *exact* distances in the *simplified* version as *heuristic* estimates in the original.

Heuristics Defined by Abstraction

- An *abstraction* of state space S is any state space $\varphi(S)$ such that:
 - for every state $s \in S$ there is a corresponding state $\varphi(s) \in \varphi(S)$.
 - $d(\varphi(s_1), \varphi(s_2)) \leq d(s_1, s_2)$.
- Exact distances in $\varphi(S)$ are consistent (and therefore admissible) heuristics for searching in S .

Example: 8-puzzle

	1	2
3	4	5
6	7	8

Domain = *blank*, 1,2,3,4,5,6,7,8

$\frac{9!}{2} = 181,440$ states

Domain Abstraction

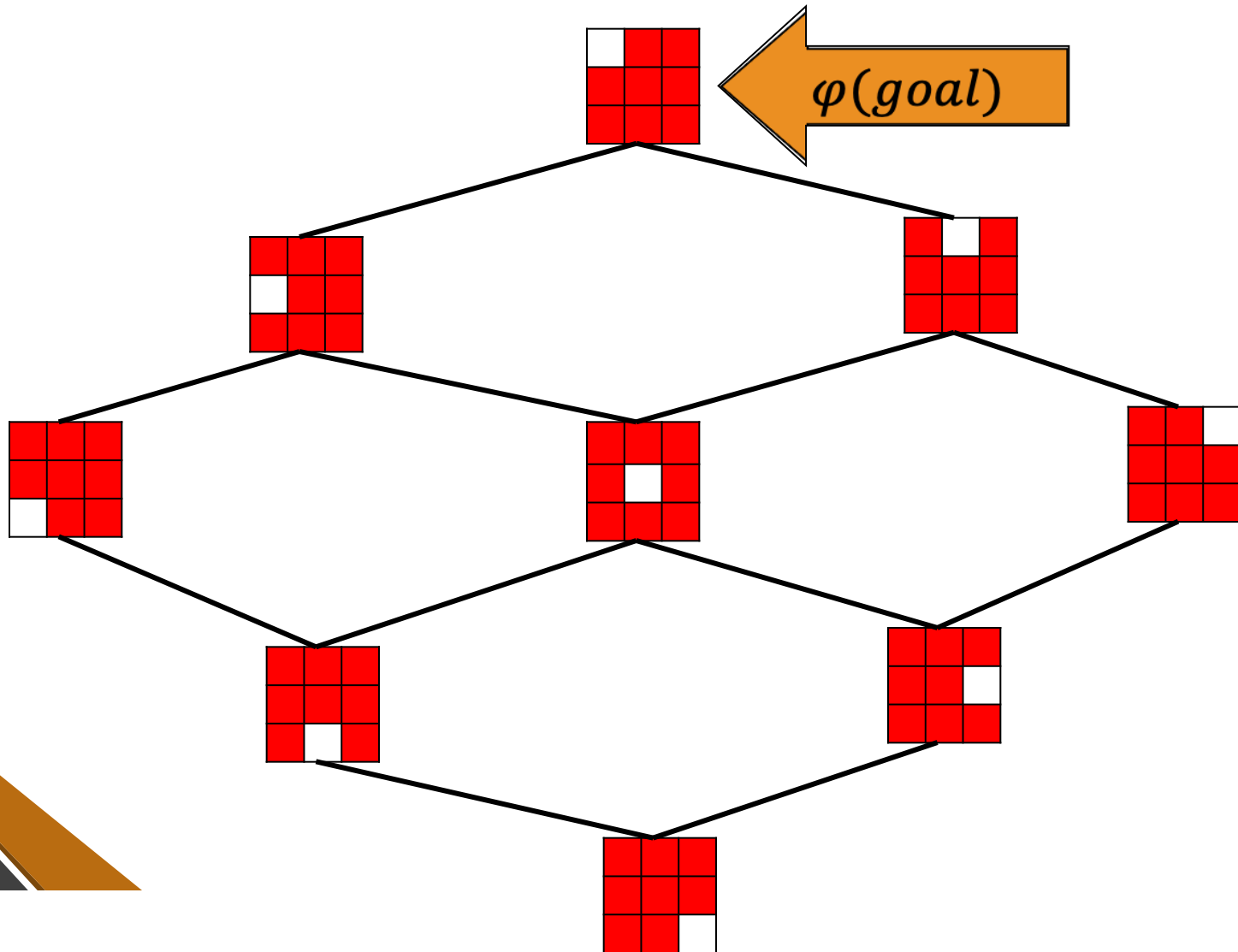
	1	2
3	4	5
6	7	8

state → **abstract state**

Domain = blank 1 2 3 4 5 6 7 8

Abstract = blank ■ ■ ■ ■ ■ ■ ■ ■

Abstract State Space

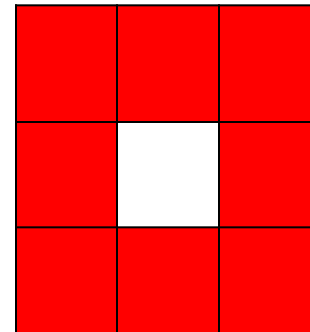


Calculating $h(s)$

Given a state $s =$

8	1	4
3		5
6	7	2

Compute the corresponding
abstract state, $\varphi(s)$



$$h(s) = d(\varphi(s), \varphi(goal)) = 2$$

Abstract Path Refinement

- A different approach: an abstract solution path is used as a skeleton
- The final solution path is obtained by “fleshing out details” and filling in gaps
- Very fast, but solutions not necessarily optimal
 - ABSTRIPS (Sacerdoti, 1974)
 - ALPINE (Knoblock, 1994)
- Potential problem: what if the abstract solution cannot be refined ?

Application to Puzzles

Rubik's Cube (Korf, 1997)

- 10^{19} states
- First random problems ever solved optimally by a general-purpose search algorithm
- Hardest took 17 CPU days
- Best known MD-like heuristic would have taken a CPU-century

15-puzzle

- 10^{13} states
- Average solution time 0.021 seconds, with only 36k nodes expanded.

Many Other Applications

- Parsing [Klein & Manning, 2003]
- Dynamic programming [Christofides et al., 1981]
- QoS network routing [Li, Harms & Holte, 2005]
- Vertex cover [Felner, Korf & Hanan, 2004]
- Cooperative pathfinding [Silver, 2005]
- Multiple sequence alignment [Korf & Zhang, 2000]
[McNaughton et al., 2002] [Zhou & Hansen, 2004]
- Planning [Edelkamp, 2001] [Bonet & Geffner, 2001]
- Constrained optimization [Kask & Dechter, 2001]

Domain Abstraction

	1	2
3	4	5
6	7	8

state → **abstract state**

Domain = blank 1 2 3 4 5 6 7 8

Abstract = blank ■ ■ ■ ■ ■ ■ ■ ■

Finer-grained Domain Abstraction

	1	2
3	4	5
6	7	8

6	7	8

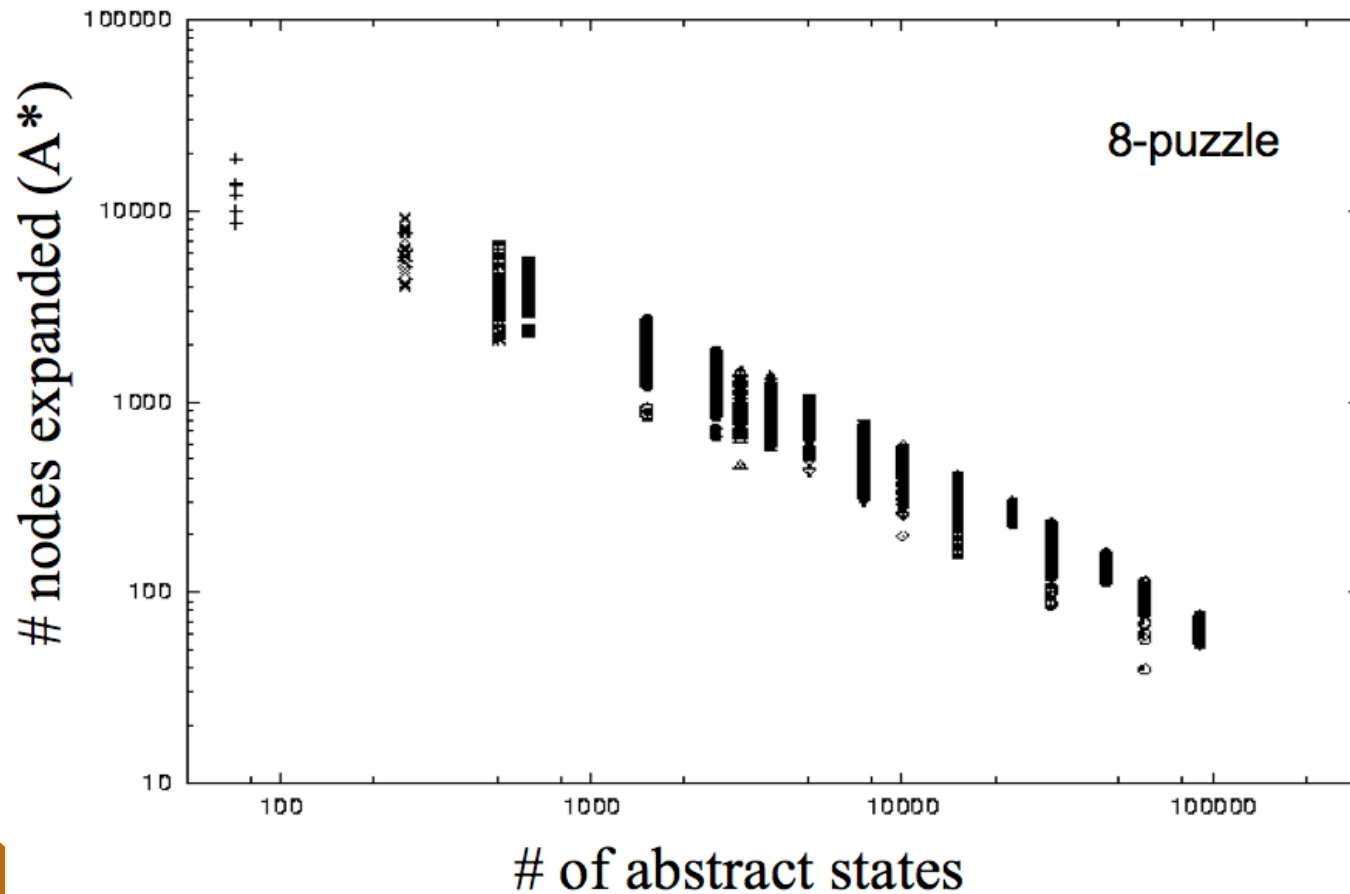
Domain = blank 1 2 3 4 5 6 7 8
Abstract = blank ■ ■ ■ ■ ■ 6 7 8

30,240 abstract states

Spoiled for Choice

- Domain abstraction always produces an admissible and consistent heuristic.
- The techniques can be used in combination with one another.
- Domain abstraction and projection produce different heuristics when applied to different encodings of the search space.

Size of Abstract Space Matters



Choosing a Good Abstraction

Some good, easy-to-compute measures:

- Average value in a Pattern Database
- The value of $h(start)$
- When there are non-identical edge costs: minimize the discrepancy in merged edge costs.

Computing Abstract Distances

- Pattern Databases
 - All possible $h(v)$ values calculated in advance, in a preprocessing step [Culberson & Schaeffer (1996)]
- Hierarchical Heuristic Search
 - $h(v)$ values calculated on demand
 - Holte et al. (1996), Hierarchical A^*
 - Holte et al. (2005), Hierarchical IDA^* .

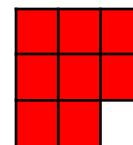
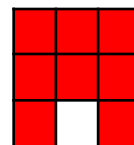
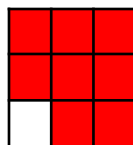
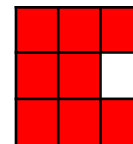
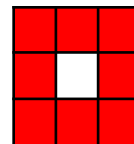
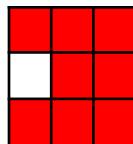
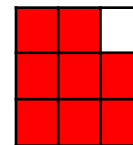
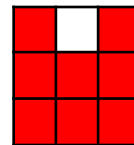
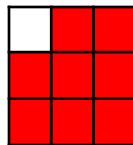
Pattern Databases

- Enumerate the entire abstract space as a preprocessing step (e.g. by BFS backwards from $\varphi(\text{goal})$).
- Store distance-to-goal for every abstract state in a lookup table (PDB).
- During Search in the original state space, $h(s)$ is computed by a lookup in the PDB.

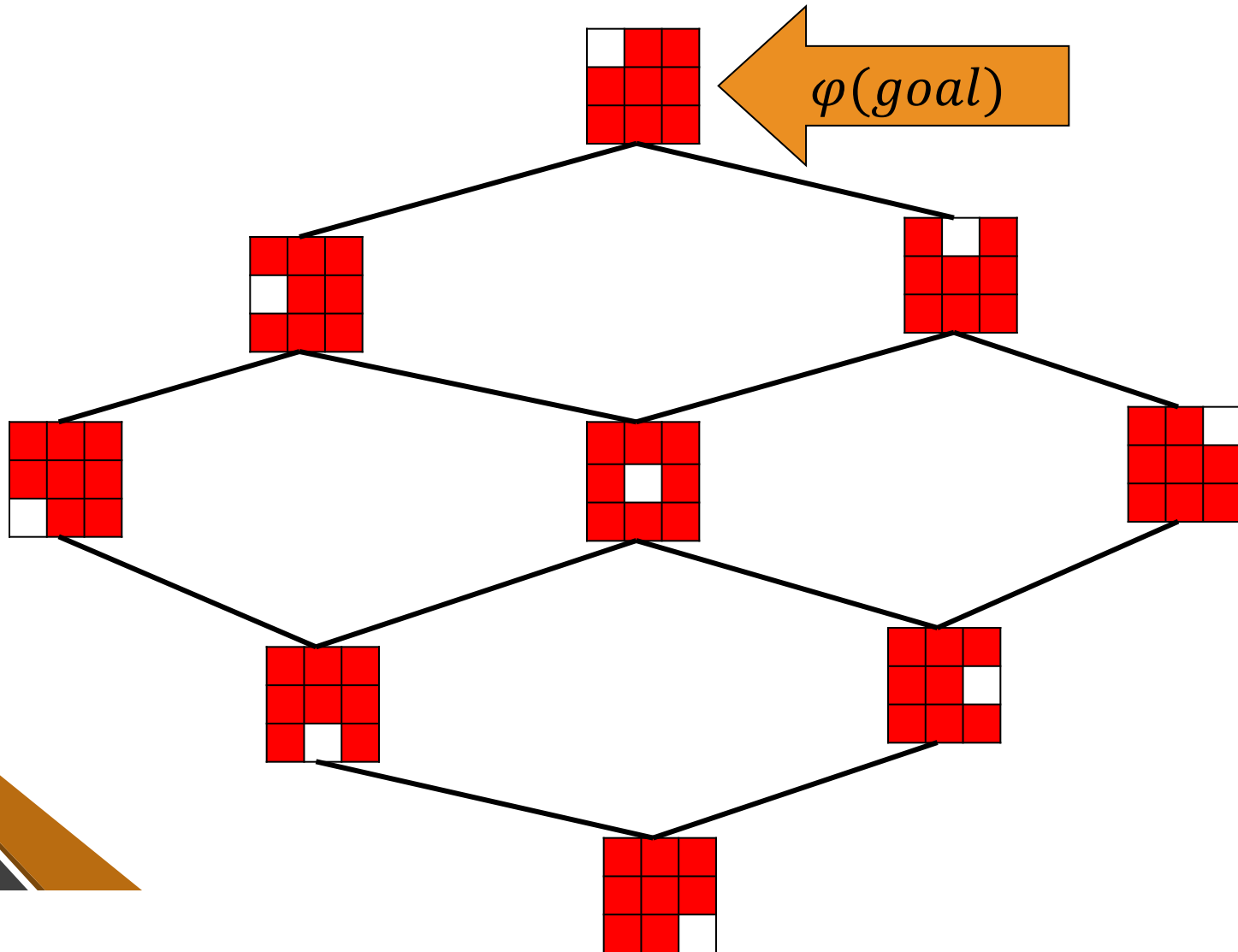
	1	2
6	4	5
7	3	8

1	2	5
6	4	8
7	3	

2		5
1	6	8
7	4	3

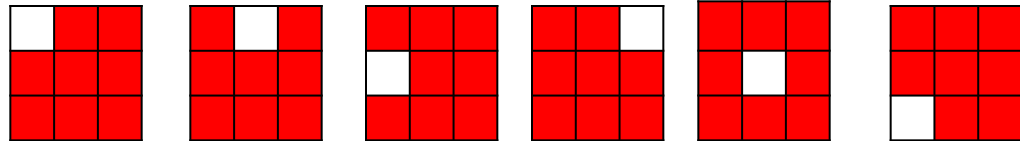


Abstract State Space



Pattern Database

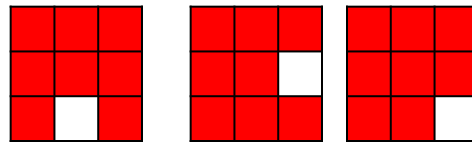
Pattern



Distance to goal

0 1 1 2 2 2

Pattern



Distance to goal

3 3 4

Hierarchical Heuristic Search

- No preprocessing.
- When $h(s)$ is needed, it is calculated by searching for a shortest path in the abstract space from $\varphi(s)$ to $\varphi(goal)$.
- Need to cache all information about abstract distance-to-goal and reuse, otherwise this will be hopelessly inefficient.

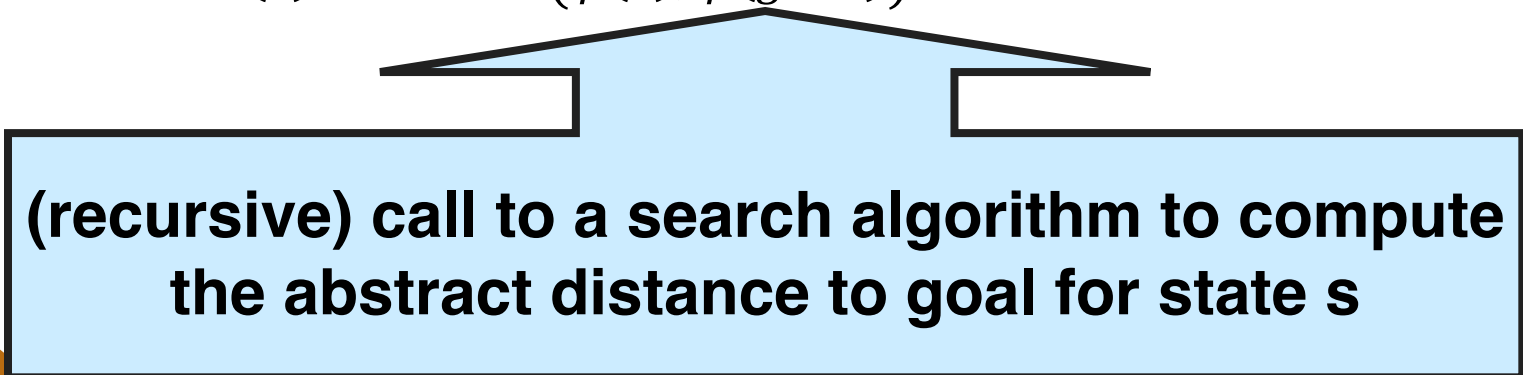
Code Comparison

PDB has this line:

$$h(s) = PDB[\varphi(s)]$$

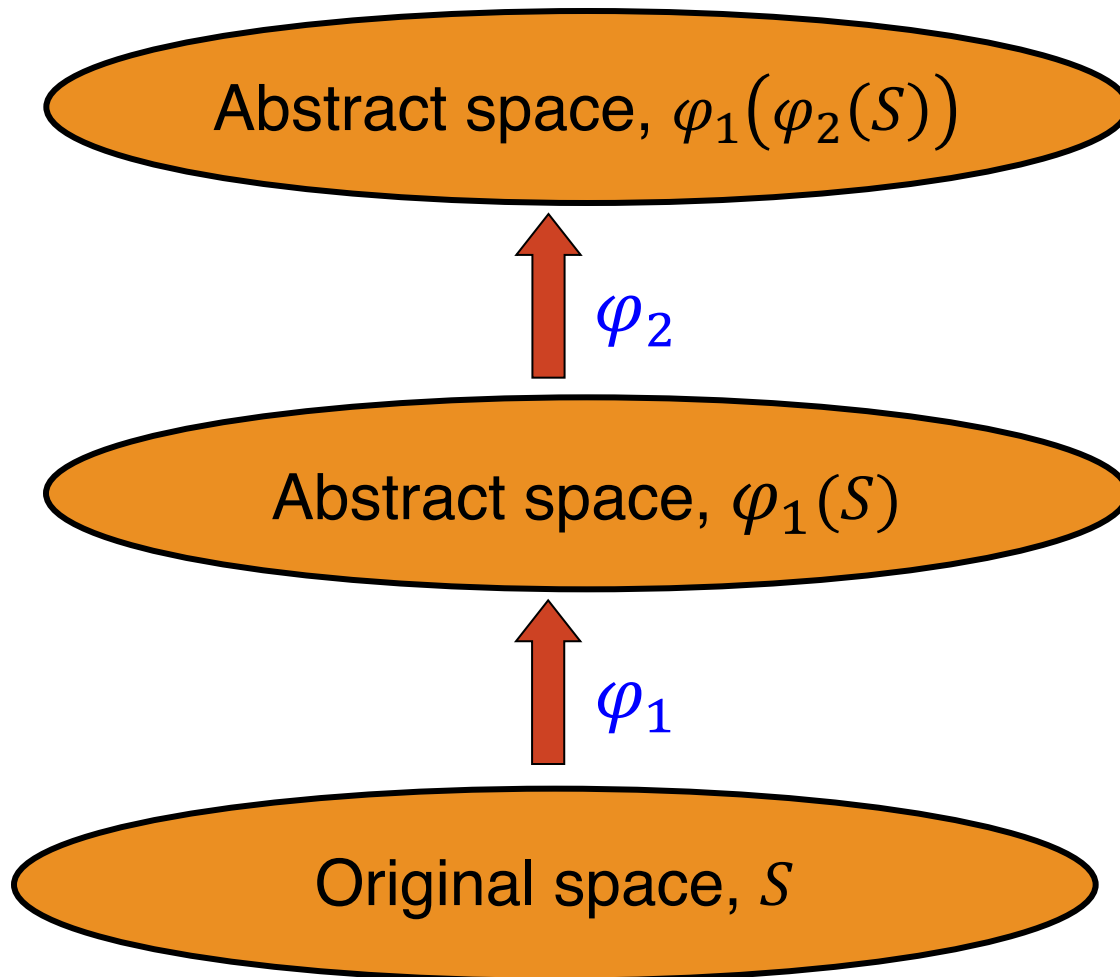
Hierarchical Heuristic Search has:

$$h(s) = search(\varphi(s), \varphi(goal))$$



(recursive) call to a search algorithm to compute the abstract distance to goal for state s

Hierarchical Heuristic Search



State-Space Abstraction

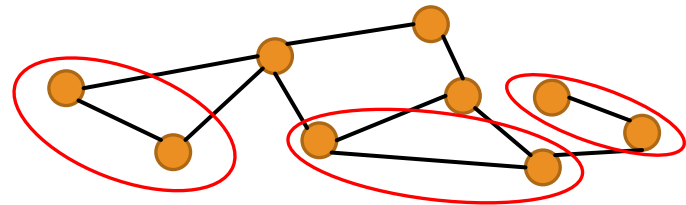
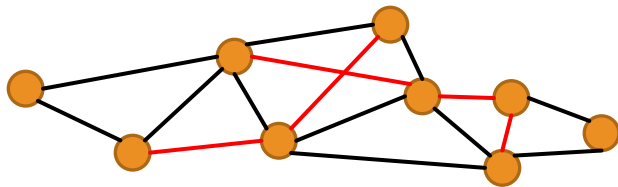
- A mapping Φ of states from state space $\langle S, d \rangle$ into $\langle S', d' \rangle$ is an abstraction transformation iff:

$$\forall s_1, s_2 \in S: d(s_1, s_2) \geq d'(\Phi(s_1), \Phi(s_2))$$

- Abstraction can be used in order to automatically create admissible heuristic functions.

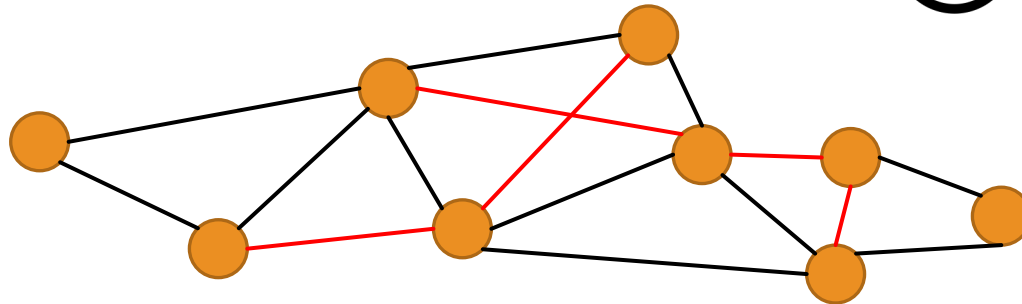
Types of Abstractions

- **Embedding** = adding edges to the original graph (corresponds to macro or relaxed operators).
- **Homomorphism** = grouping together sets of states to create a single abstract state (corresponds to dropping a feature or variable from the state representation).



Not always a useful idea...

The total cost of computing h over the course of the search can exceed the savings.



Gaschnig (1979)

- *Estimate* the cost of solutions in S by the *exact* cost of solutions in auxiliary space T .
- Estimates are admissible if T is an edge supergraph of S .
- T should be supplied with an efficient solver!

“If T is solved by searching this could consume more time than solving in S directly with breadth-first search.”

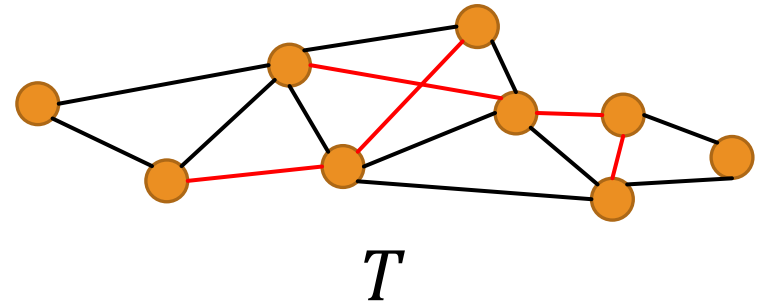
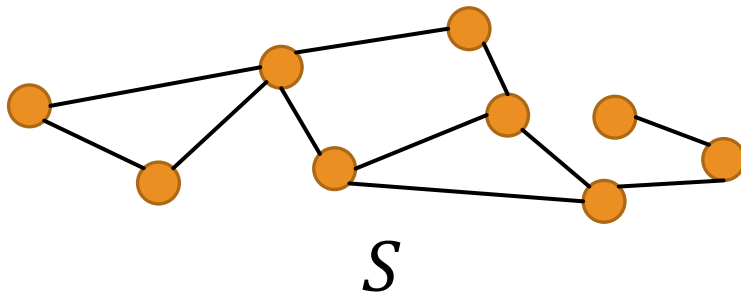
Valtorta (1980,1984) Proved that Gaschnig was Right!

- T is an edge supergraph of S .
- Distances in T are computed by BFS.
- We use distances in T to compute an A^* heuristic to solve problem over S .

Use A^* with distances in T



Use BFS to compute distances in T

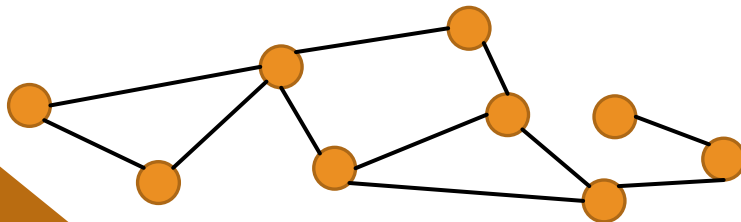


Valtorta (1980,1984) Proved that Gaschnig was Right!

Theorem: if we expand a node s in S using BFS, then the node s will be expanded either when we search through T or when we search via A^* .

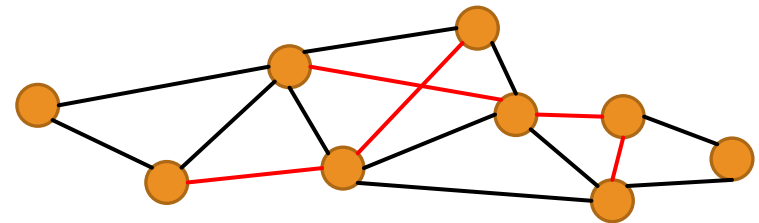
In other words, # nodes checked using BFS $\leq$ # of nodes checked when searching via a heuristic computed via iterating over T using BFS.

Use A^* with distances in T



S

Use BFS to compute distances here



T

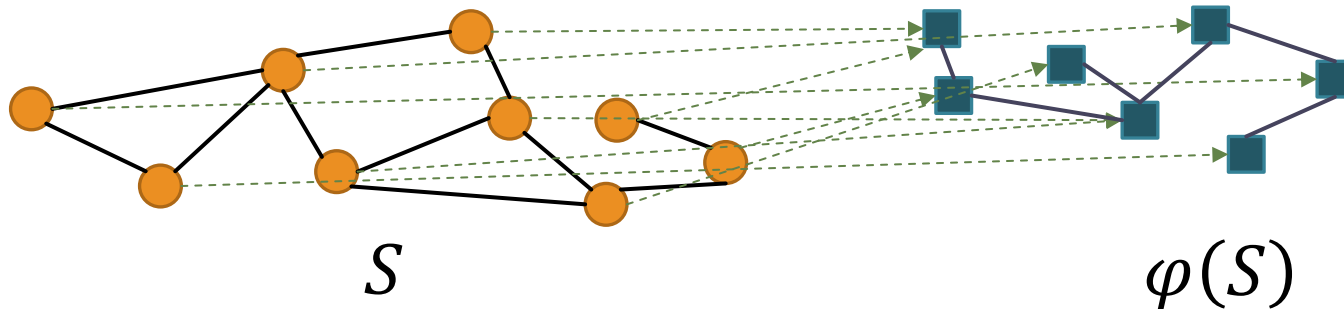
Generalized Valtorta's Theorem

- $\varphi(S)$ is an abstraction of S .
- Distances in $\varphi(S)$ are computed by BFS.
- We use distances in $\varphi(S)$ to compute an A^* heuristic to solve problem over S .

Use A^* with distances in $\varphi(S)$



Use BFS to compute distances in $\varphi(S)$

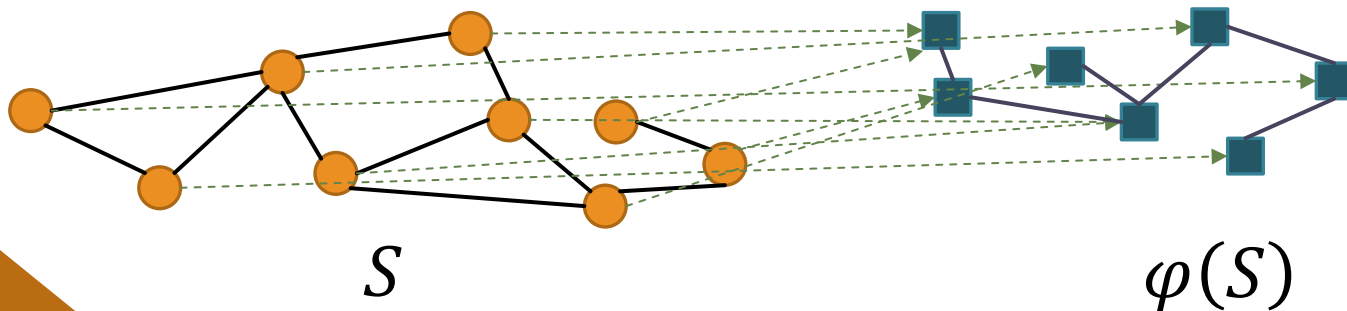


Generalized Valtorta's Theorem

Theorem: if we expand a node s in S using BFS, then either $\varphi(s)$ is expanded when we search through $\varphi(S)$ or s is checked when we search S using A^* .

No speedup when φ is an embedding.

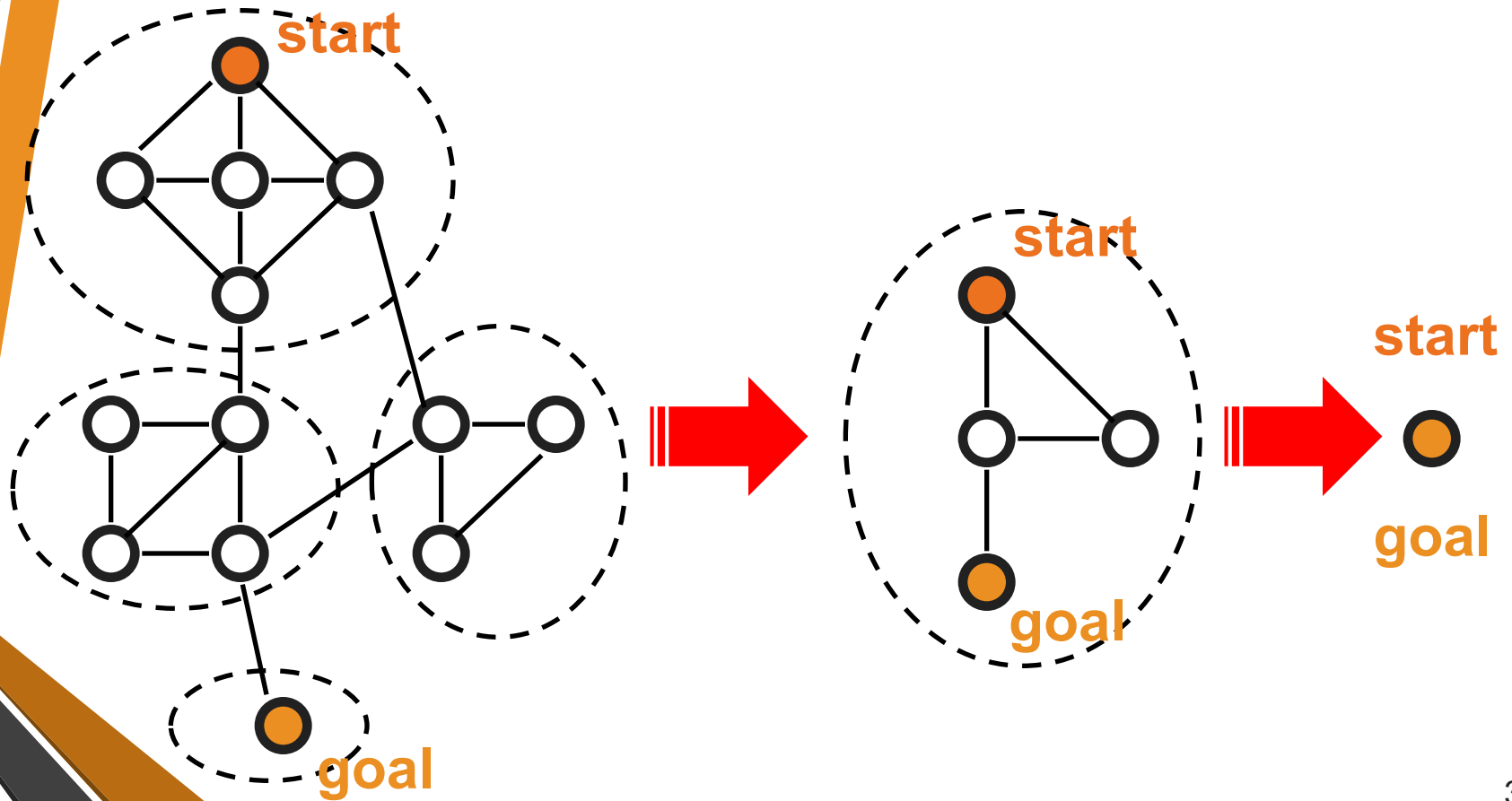
Possible speedup when φ is a homomorphism: if lots of states get mapped to the same abstract state, we can end up saving a lot of time.



What Speedup Can be Achieved?

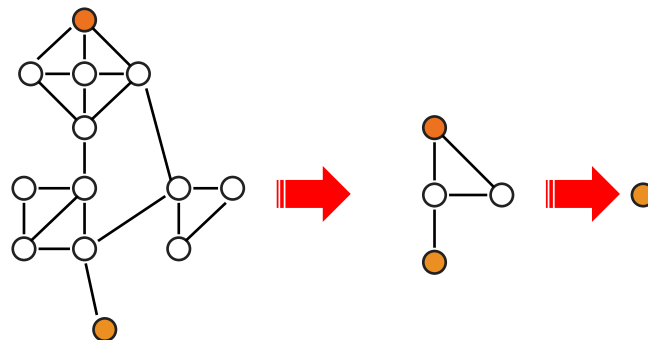
- Holte et al. (1996) tried to answer this question for “Max-degree” star abstraction
- The state with the highest degree is grouped together with its neighbors within a certain distance (the abstraction radius) to form a single abstract state.

Star Abstraction with Radius = 2



Naive Hierarchical A^*

- Like A^* , except that $h(s)$ is computed by searching at the next higher level of abstraction.
- The result is combined with other estimates (e.g., cheapest operator cost) to produce the final $h(s)$.
- Heuristic values are cached to improve performance.



Naive Hierarchical A*

TABLE 1. Naive Hierarchical A*. (abstraction radius = 2)

Search Space	Size (# states)		Nodes Expanded		
	All Levels	Base Level	Blind Search	Hierarchical A* All Levels	Base Level
Blocks-5	1166	866	389	2766	118
5-puzzle	961	720	348	3119	224
Fool's Disk	4709	4096	1635	12680	629
Hanoi-7	2894	2187	1069	18829	701
KL2000	3107	2736	1236	7059	641
MC 60-40-7	2023	1878	934	2412	702
Permute-6	731	720	286	806	77
Words	5330	4493	1923	19386	604

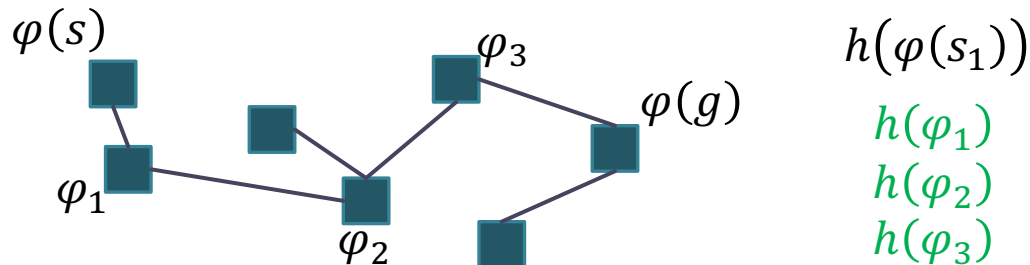
Reducing Search Effort

Observation: all searches related to the same base level problem have the same goal.

- This allows additional types of caching of values.
- Breaks Valtorta's barrier in 5 out of 8 search spaces.

Exploiting Information Recovered During Blind Search Through Abstract Space

- V1: h^* caching – cache exact distances to goal along discovered optimal paths in abstract space.



- V2: cache optimal path in abstract space + h value.
- V3: for any abstract state φ_j *not* on solution path, we know the path length from $\varphi(s)$ to φ_j , $g(\varphi_j)$. We also know optimal path length P : store the value $P - g(\varphi_j)$.

Hierarchical A*

TABLE 2. Hierarchical A*. (abstraction radius = 2)

TABLE 2. Hierarchical A*. (abstraction radius = 2)						
Search Space	Blind Search	Nodes Expanded				# problems V3 < BS (out of 200)
		Hierarchical A*				
		Naive	V1	V2	V3	
Blocks-5	389	2766	1235	478	402	96
5-puzzle	348	3119	1616	854	560	14
Fool’s Disk	1635	12680	8612	3950	1525	132
Hanoi-7	1069	18829	10667	5357	3174	0
KL2000	1236	7059	3490	1596	1028	171
MC 60-40-7	934	2412	1531	1154	863	128
Permute-6	286	806	482	279	242	113
Words	1923	19386	7591	2849	1410	124

The Granularity of Abstraction

- Increasing the radius of abstraction has two contradictory effects:
 - + **abstract spaces contain fewer states, and each abstract search produces values for more states, but**
 - **the heuristic is less discriminating**
- The best case breaks the Valtorta's barrier in every search space.

A* with Best Abstraction Radius

TABLE 3. Hierarchical A*. (best abstraction radius)

Search Space	Radius	Nodes Expanded			# problems V3 < BS (out of 200)	CPU seconds	
		Blind Search	Hierarchical A* Naive	V3		Blind Search	V3
Blocks-5	5	389	611	309	123	69	86
5-puzzle	12	348	354	340	131	36	40
Fool's Disk	4	1635	1318	1172	194	872	902
Hanoi-7	20	1069	1097	1055	117	102	108
KL2000	5	1236	1306	1072	178	398	384
MC 60-40-7	4	934	822	803	144	266	253
Permute-6	5	286	201	194	192	82	67
Words	3	1923	9184	1356	128	1169	1273

PDB vs HHS - Time

- Pattern Databases
 - Large preprocessing time
 - 15-puzzle: 2.5 hours*
 - TopSpin: 40 minutes*
 - Very fast $h(s)$ computation during search
 - 15-puzzle instance solved in 0.022 seconds (avg)
- Hierarchical Heuristic Search
 - No preprocessing time
 - Relatively slow $h(s)$ computation

* Times are for the best-performing PDBs. Smaller PDBs take less time to build but take correspondingly longer to solve problems.

PDB vs HHS - Memory

- Pattern Databases
 - Perfect hash function
 - No empty hash table entries
 - Each entry stores only a distance (15-puzzle: 1 byte)
 - Only a tiny fraction of entries needed to solve an individual search problem
- Hierarchical Heuristic Search
 - Imperfect hash function (15-puzzle: 8 bytes)
 - Multiple levels of abstraction, not just one
 - Only store entries needed to solve the given problem

When to Use Each Approach?

- If the same abstraction can be used to solve many problems, use PDB.
- If there is only one problem to solve, or a small batch of problems, use Hierarchical Heuristic Search.

Max'ing Multiple Heuristics

- Given heuristics h_1 and h_2 define

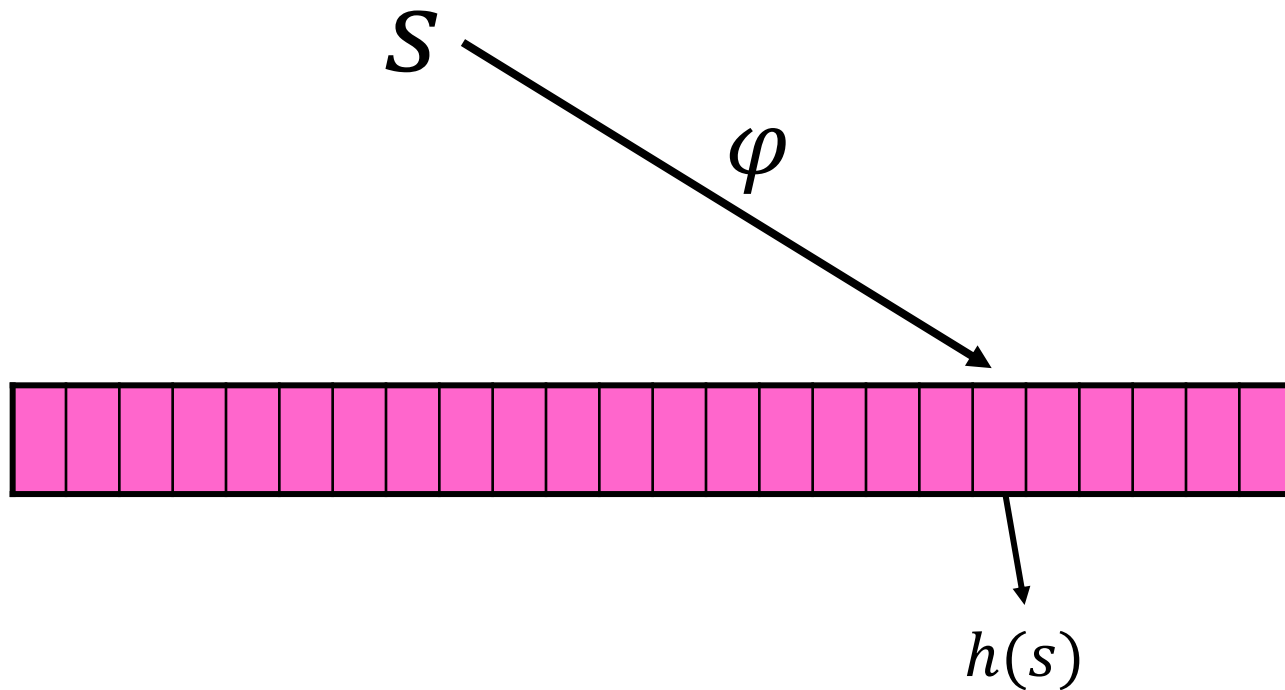
$$h(s) = \max(h_1(s), h_2(s))$$

- Preserves key properties:
 - lower bound
 - consistency

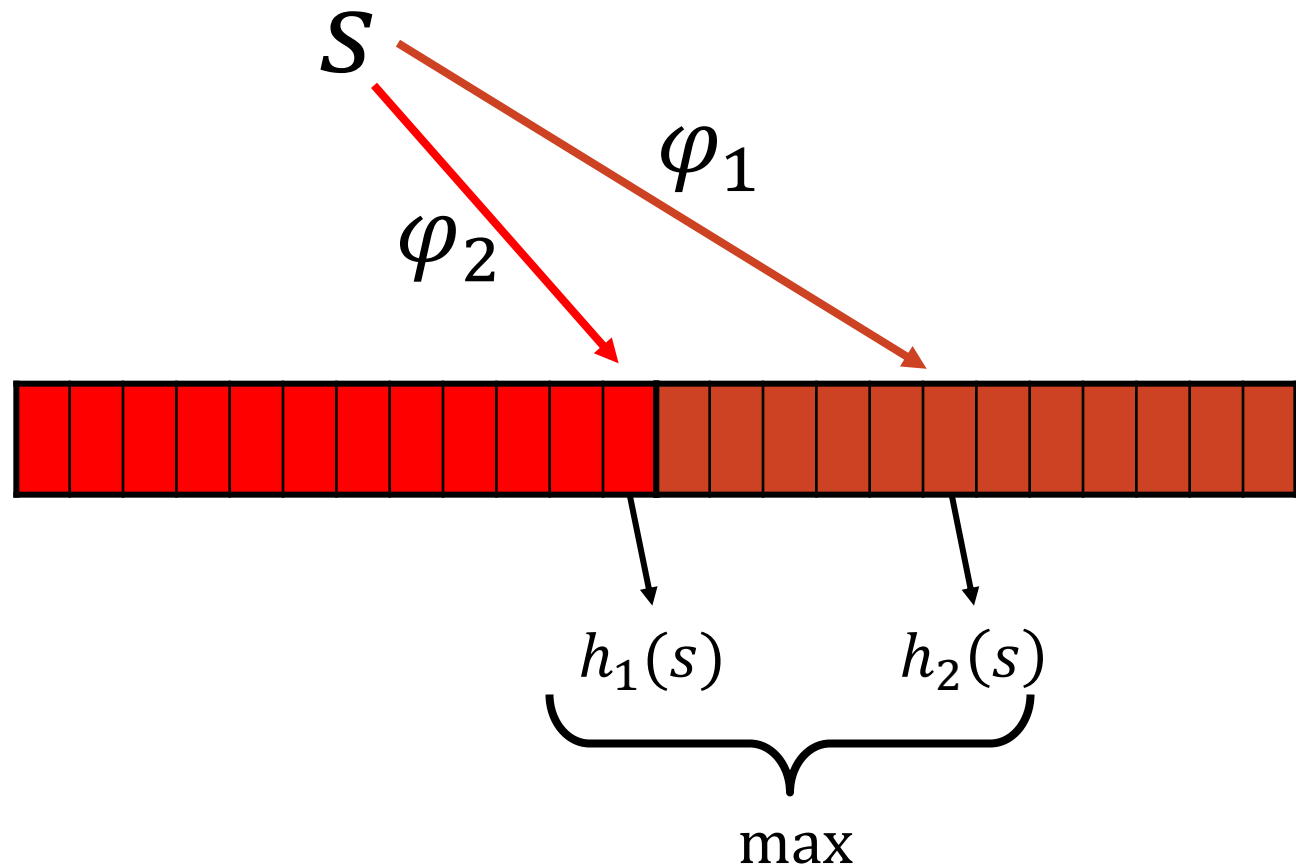
Question

- Given a fixed amount of memory, M , which gives the best heuristic ?
 - 1 pattern database (PDB) of size M
 - max'ing 2 PDBs of size $\frac{M}{2}$
 - max'ing 3 PDBs of size $\frac{M}{3}$
 - etc.

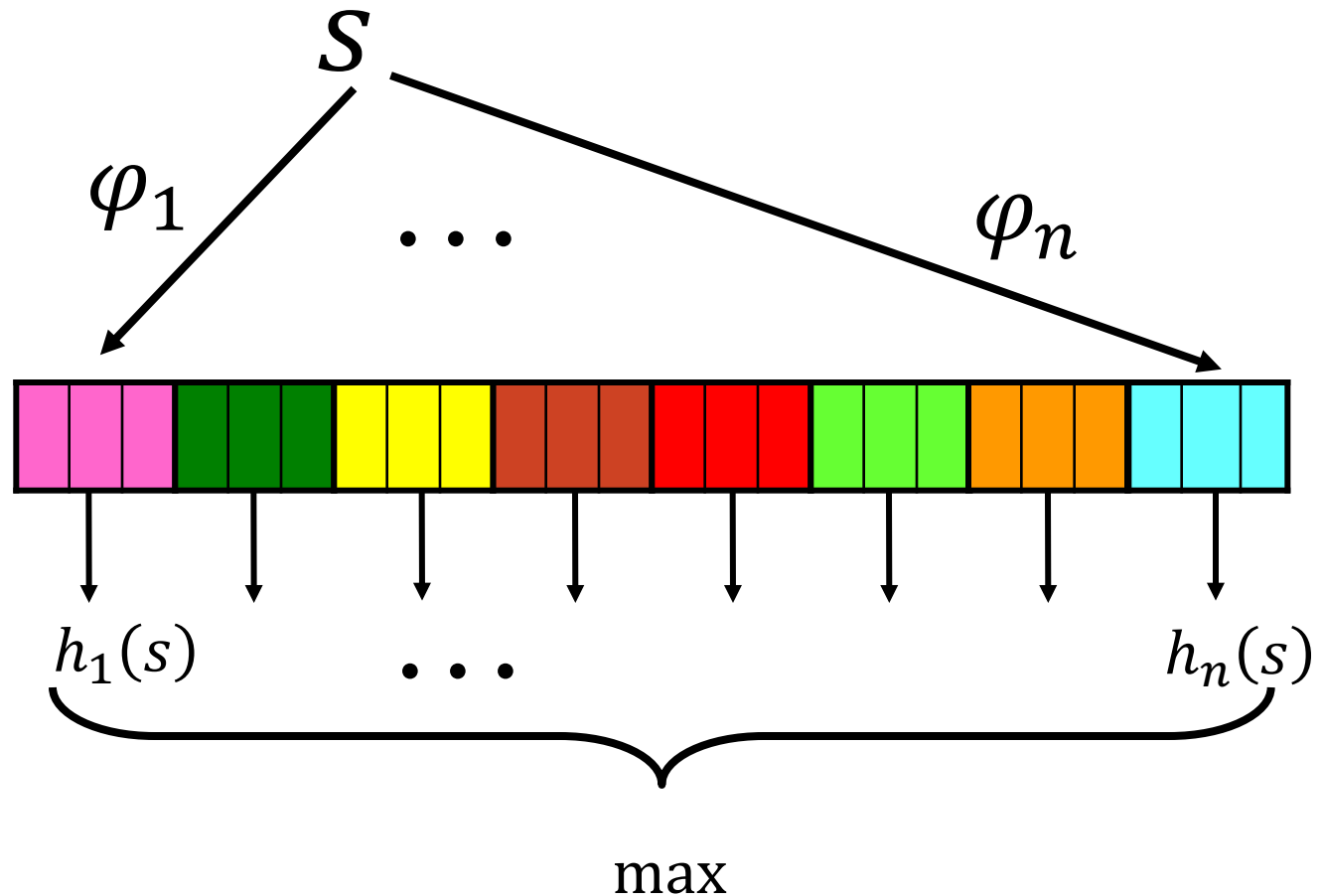
One large pattern database



Two half-size pattern databases



Many small pattern databases



Rubik's Cube*

PDB Size	n	Nodes Generated
13,305,600	8	2,654,689
17,740,800	6	2,639,969
26,611,200	4	3,096,919
53,222,400	2	5,329,829
106,444,800	1	61,465,541

* “easy” problems

$$RATIO = \frac{\# \text{ nodes generated using one PDB of size } M}{\# \text{ nodes generated using } n \text{ PDBs of size } \frac{M}{n}}$$

State Space	Best n	Ratio
(3x3)-puzzle	10	3.85
9-pancake	10	8.59
(8,4)-Topspin (3 ops)	9	3.76
(8,4)-Topspin (8 ops)	9	20.89
(3x4)-puzzle	21+	185.5
Rubik's Cube	6	23.28
15-puzzle (additive)	5	2.38
24-puzzle (additive)	8	1.6 to 25.1

Rubik's Cube CPU Time

#PDBs	Nodes Ratio	Time Ratio
8	23.15	12.09
6	23.28	14.31
4	19.85	13.43
2	11.53	9.87
1	1.00	1.00

time/node is 1.67x higher using **six** PDBs

Adding instead of Max'ing

- Under some circumstances it is possible to add the values from two PDBs instead of just max'ing them and still have an admissible heuristic.
- This is advantageous because

$$h_1(s) + h_2(s) \geq \max(h_1(s), h_2(s))$$

Manhattan Distance Heuristic

For a sliding-tile puzzle, Manhattan Distance looks at each tile individually, counts how many moves it is away from its goal position, and adds up these numbers.

	1
2	3

goal

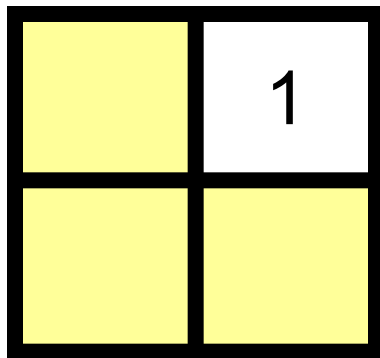
3	
1	2

state s

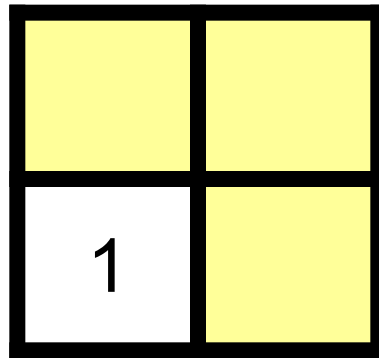
$$MD(s) = 2 + 1 + 2 = 5$$

M.D. as Additive PDBs (1)

$$\varphi_1(x) = \begin{cases} x & \text{if } x = 1 \\ \text{blank} & \text{otherwise} \end{cases}$$



$\varphi_1(goal)$



$\varphi_1(s)$

$$PDB_1[\varphi_1(s)] = 2$$

$$MD[s] = PDB_1[\varphi_1(s)] + PDB_2[\varphi_2(s)] + PDB_3[\varphi_3(s)]$$



In General...

$$\varphi_1(x) = \begin{cases} x & \text{if } x \in G_i \\ \textit{blank} & \text{otherwise} \end{cases}$$

Partition the tiles in groups $G_1, \dots, G_k$

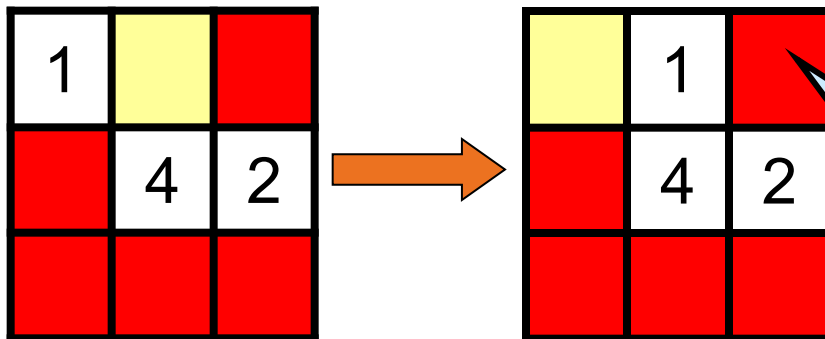
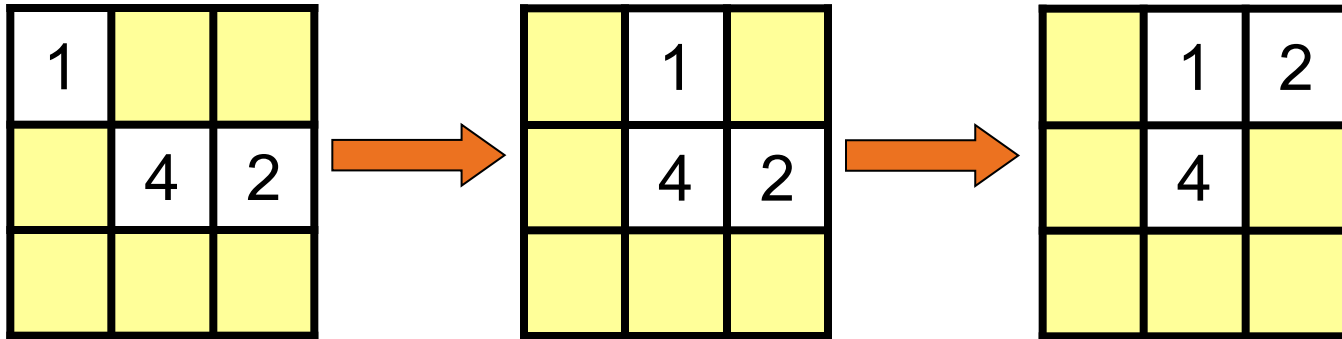
Korf & Felner's Method

$$\varphi_i(x) = \begin{cases} x & \text{if } x \in G_i \\ blank & \text{if } x = blank \\ \blacksquare & \text{otherwise} \end{cases}$$

Partition the tiles into $G_1, \dots, G_k$

Moves of $\blacksquare$ cost zero

What's the Difference ?



the blank cannot reach this position without disturbing tile 1 or tile 2.

Compared to Max'ing

- If the PDBs were going to be max'd instead of added, we would count all the moves in all the PDBs.
- Therefore, the PDBs for adding have smaller entries than the corresponding PDBs for max'ing.
- In initial experiments on the 15-puzzle, max'ing returns a higher value than adding for about 12% of the states.